pNodeReporter and uSimMarineV22

Node reporting and vehicle simulation.

Included MIT MOOS-IvP PDF sources:

- app_pnreporter.pdf
- app_usimmarine_v22.pdf

pNodeReporter: Summarizing a Node's Position and Status

Feb 2025

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/appdocs/app_pnreporter`

1 Overview

The `pNodeReporter` app runs on each vehicle (real or simulated) and generates node reports for sharing between vehicles, depicted in Figure 1. They can be regarded as a proxy for AIS reports. The app serves one primary function - it repeatedly gathers local platform information and navigation data and creates an AIS like report in the form of the MOOS variable `NODE_REPORT_LOCAL`. Node reports are communicated between the vehicles and the shore or shipside command and control through an inter-MOOSDB communications process such as `pShare` or via acoustic modem. Since a node or platform may both generate and receive reports, the locally generated reports are labeled

with the _LOCAL suffix and transmitted to the outside communities without the suffix. This is to ensure that processes running locally may easily distinguish between locally generated and externally generated reports.

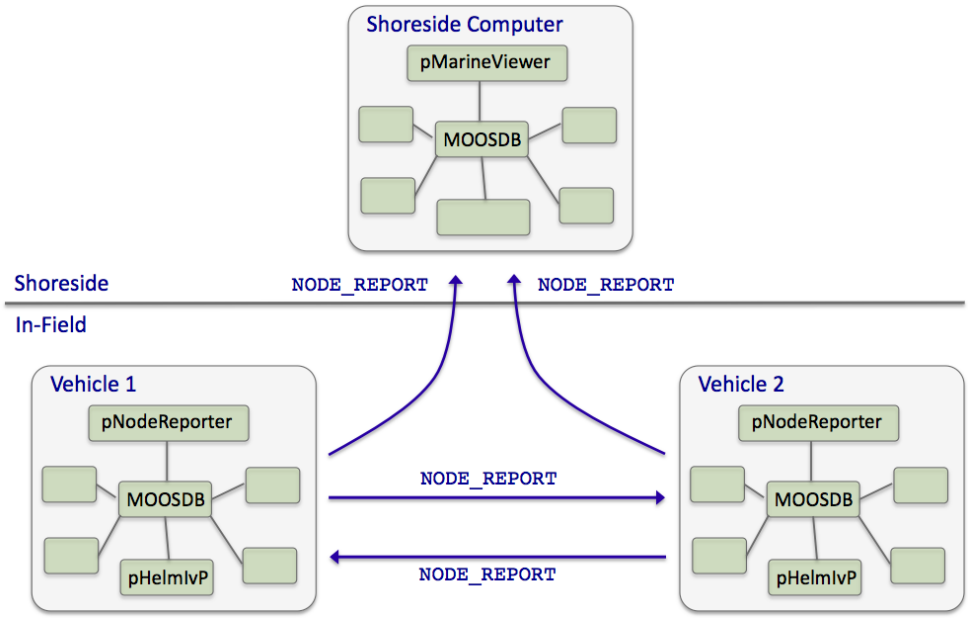

Figure 1: **Typical pNodeReporter usage:** The pNodeReporter application is typically used with pShare or acoustic modems to share node summaries between vehicles and to a shoreside command-and-control GUI.

To generate the local report, pNodeReporter registers for the local NAV_* vehicle navigation data and creates a report in the form of a single string posted to the variable NODE_REPORT_LOCAL. An example of this variable is given in below in Section 2.1. The pMarineViewer and pHelmIvP applications are two modules that consume and parse the incoming NODE_REPORT messages.

The pNodeReporter utility may also publish a second report, the PLATFORM_REPORT. While the NODE_REPORT summary consists of an immutable set of data fields described later in this section, the PLATFORM_REPORT consists of data fields configured by the user and may therefore vary widely across applications. The user may also configure the frequency in which components of the PLATFORM_REPORT are posted within the report.

2 Using pNodeReporter

2.1 Overview Node Report Components

The primary output of pNodeReporter is the node report string. It is a comma-separated list of key-value pairs. The order of the pairs is not significant. The following is an example report:

```
NODE_REPORT_LOCAL = "NAME=alpha,TYPE=UUV,TIME=1252348077.59,X=51.71,Y=-35.50,
                     LAT=43.824981,LON=-70.329755,SPD=2.00,HDG=118.85,YAW=118.84754,
```

DEP=4.63,LENGTH=3.8,MODE=MODE@ACTIVE:LOITERING"

The TIME reflects the Coordinated Universal Time as indicated by the system clock running on the machine where the MOOSDB is running. Speed is given in meters per second, heading is in degrees in the range $[0, 360)$, depth is in meters, and the local x-y coordinates are also in meters. The source of information for these fields is the NAV_* navigation MOOS variables such as NAV_SPEED. The report also contains several components describing characteristics of the physical platform, and the state of the IvP Helm, described next.

If desired, pNodeReporter may be configured to use a different variable than NODE_REPORT_LOCAL for its node reports, by setting the configuration parameter node_report_output to MY_REPORT for example. Most applications that subscribe to node reports, subscribe to two variables, NODE_REPORT_LOCAL and NODE_REPORT. This is because node reports are meant to be bridged to other MOOS communities (typically with pShare but not necessarily). A node report should be broadcast only from the community that generated the report. In practice, to ensure that node reports that arrive in one community are not then sent out to other communities, the node reports generated locally have the _LOCAL suffix, and when they are sent to other communities they are sent to arrive with the new variable name, minus the suffix.

2.2 Helm Characteristics

The node report contains one field regarding the current mode of the helm, MODE. Typically the pNodeReporter and pHelmIvP applications are running on the same platform, connected to the same MOOSDB. When the helm is running, but disengaged, i.e., in manual override mode, the MODE field in the node report simply reads "MODE=DISENGAGED". When or if the helm is detected to be not running, the field reads "MODE=NOHELM-SECS", where SECS is the number of seconds since the last time pNodeReporter detected the presence of the helm, or "MODE=NOHELM-EVER" if no helm presence has ever been detected since pNodeReporter has been launched.

How does pNodeReporter know about the health or status of the helm? It subscribes to two MOOS variables published by the helm, IVPHELM_STATE and IVPHELM_SUMMARY. These are described more fully in helm documentation, but below are typical example values:

```
IVPHELM_STATE   = "DRIVE"

IVPHELM_SUMMARY = "iter=72,ofnum=1,warnings=0,time=127349406.22,solve_time=0.00,
                   create_time=0.00,loop_time=0.00,var=course:209.0,var=speed:1.2,
                   halted=false,running_bhvs=none,modes=MODE@ACTIVE:LOITERING,
                   active_bhvs=loiter$17.8$100.00$9$0.04$0/0,completed_bhvs=none
                   idle_bhvs=waypt_return$17.8$0/0:station-keep$17.8$n/a
```

The IVPHELM_STATE variable is published on each iteration of the pHelmIvP process regardless of whether the helm is in manual override ("PARK") mode or not, and regardless of whether the value of this variable has changed between iterations. It is considered the "heartbeat" of the helm. This is the variable monitored by pNodeReporter to determine whether a "NOHELM" message is warranted. By default, a period of five seconds is used as a threshold for triggering a "NOHELM" warning. This value may be changed by setting the nohelm_threshold configuration parameter.

When the helm is indeed engaged, i.e., not in manual override mode, the value of IVPHELM_STATE posting simply reads "DRIVE", but the helm further publishes the IVPHELM_SUMMARY variable similar to the above example. If the user has chosen to configure the helm using hierarchical mode declarations (as described in helm documentation), the IVPHELM_SUMMARY posting will include a component such as "modes=MODE@ACTIVE:LOITERING" as above. This value is then included in the node report by pNodeReporter. If the helm is not configured with hierarchical mode declarations, the node report simply reports "MODE=DRIVE".

2.3 Custom Rider Augmentation of the Node Report

A node report typically is limited to the components shown in the Section 2.1, e.g., vehicle heading, speed, and position. These components are derived from the MOOS variables NAV_HEADING, NAV_SPEED, NAV_LAT and NAV_LONG.

A rider is a custom augmentation of the node report, resulting in additional param=value pairs in the outgoing node report. There are *static* riders, configured in the mission file, and *dynamic* riders with values filled in at run time, where the source is a named MOOS variable, configured in the mission file.

2.3.1 Static Riders

A *static* rider is set in the mission file, typically for values that are known prior to launch time, and do not change during the course of the mission. Post Release 24.8, this is possible with the platform_aspect configuration parameter: For example:

```
platform_aspect = ID=439
platform_aspect = FUEL_TYPE=diesel
```

This will result in the string "ID=439,FUEL_TYPE=diesel" added to all outgoing node reports.

Of course the following is possible too:

```
platform_aspect = ID=$(ID)
```

In this case presumably the $(ID) macro is filled in at launch time with a utility like nsplug.

Note: if two platform_aspect configurations are made with the same lefthand component, the second line will overwrite the first, and there will be no warning.

2.3.2 Dynamic Riders

A *dynamic* rider is a param=value component where the value changes dynamically at run time. For example, if there are MOOS variables ENGINE_TEMP, or FUEL_LEVEL, a user may wish to include these in a node report. The last part of the node report may look like ETMP=145.32,FUEL=84.5.

Post Release 24.8, this can be done with the rider configuration parameter. Each rider contains (a) a MOOS variable, (b) the field name in the node report, (c) the frequency at which it is include

in node reports, and (d) the precision, if the value is numerical. Continuing with the above two examples, the configuration would like:

```
rider = var=ENGINE_TEMP, rfld=ETMP, policy=always, prec=2
rider = var=FUEL_LEVEL, rfld=FUEL, policy=always, prec=1
```

The `var` component names the MOOS variable to watch. The `rfld` component names the "rider field", i.e., the field in the node report. The policy component indicates how often it should be included in the node report. If set to *always*, it will be in every report. If set to *updated*, it will be include in reports only when the value changes. If set to *15*, it will be included only once every 15 seconds. If set to *updated+15*, it will be included immediately if the value is updated, or otherwise once every 15 seconds. The `prec` component names the precision if the value is numerical. A value of say 3, will be posted with three digits after the decimal.

2.4 Enabling Sparse Node Report Intervals

By default, a node report is generated on each iteration of pNodeReporter. In certain situations a reduction in the frequency may be appropriate. Each node report contributes both to the local alog file size, and consumes inter-vehicle communications bandwidth. If the vehicle is traversing a long straight leg, with little to no deviation in heading or speed, the vehicle position can be inferred by extrapolation. The node report frequency can perhaps be reduced from say 4Hz, to 0.5Hz.

Extrapolation is used in two parts:

- The sender of reports, pNodeReporter, extrapolates from the last posted report, for the time since the last posted report, and compares the difference. If the difference between the extrapolated position and the actual current position exceeds a configured threshold, then a node report with the current position is posted.
- The receiver of reports, e.g., other vehicles, or pMarineViewer, or similar, use extrapolation from the last received node report to update local vehicle position state.

The following configuration parameters may be used to reduce the frequency of node reports:

The extrap_enabled parameter must set to `true` to enable reduced posting frequency through extrapolation. By default it is set to `false`.

The extrap_pos_thresh parameter is a value given in meters. The current vehicle position is compared against the position extrapolated from the last reported position assuming the heading in the last reported position, and the time since the last reported position. If the distance between these two positions is greater than this threshold, a node report is generated regardless of other configuration parameters.

The extrap_hdg_thresh parameter is a value given in degrees. The current vehicle heading is compared to the heading in the most recently posted node report. If the difference in heading exceeds the threshold, then a node report is generated regardless of other configuration parameters.

The extrap_max_gap parameter is a value given in seconds. The current time is compared to the time of the most recently posted node report. If the difference in time exceeds the threshold, then a

node report is generated regardless of other configuration parameters.

The default values for these configuration parameters are:

```
extrap_enabled    = false
extrap_pos_thresh = 0.25    // meters
extrap_hdg_thresh = 1       // degrees
extrap_max_gap    = 5       // seconds
```

2.5 Platform Characteristics

The node report contains four fields regarding the platform characteristics, NAME, TYPE, LENGTH, and BEAM. The name of the platform is equivalent to the name of the MOOS community within which pNodeReporter is running. The MOOS community is declared as a global MOOS parameter (outside any given process' configuration block) in the .moos mission file. The TYPE, LENGTH, and BEAM parameters are set in the pNodeReporter configuration block.

Example:

```
platform_length = 16              // meters
platform_beam   = 15              // meters
platform_type   = kayak
platform_color  = dodger_blue
```

If the platform type is known, but no information about the platform length is known, certain rough default values may be used if the platform type matches one of the following: "kayak" maps to 4 meters, "mokai" maps to 4 meters, "uuv" maps to 4 meters, "auv" maps to 4 meters, "ship" maps to 18 meters, "glider" maps to 3 meters.

2.6 Dealing with Local versus Global Coordinates

A primary component of the node report is the current position of the vehicle. The pNodeReporter application subscribes for the following MOOS variables to garner this information: NAV_X, NAV_Y in local coordinates, and the pair NAV_LAT, NAV_LONG in global coordinates. These two pairs should be consistent, but what if they are not? And what if pNodeReporter is receiving mail for one pair but not the other? Four distinct policy choices are supported, set in the configuration parameter crossfill_policy:

- The default policy: node reports include exactly what is given. If NAV_X and NAV_Y are being received only, then there will be no entry in the node report for global coordinates, and vice versa. If both pairs are being received, then both pairs are reported. No attempt is made to check or ensure that they are consistent. This is the default policy, equivalent to the configuration crossfill_policy=literal.
- If one of the two pairs is not being received, pNodeReporter will fill in the missing pair from the other. This policy can be chosen with the configuration cross_fill_policy=fill-empty.

- If NAV_LAT and NAV_LONG are being received, pNodeReporter will use these in its report, *and* convert the global coordinates to local coordinates, and use these converted coordinates in its report, essentially disregarding NAV_X and NAV_Y if they are also being received. This policy can be chosen with the configuration cross_fill_policy=global. This is available in the next release after Release 19.8.x.
- If one of the two pairs has been received more recently, the older pair is updated by converting from the other pair. The older pair may also be in a state where it has never been received. This policy can be chosen with the configuration cross_fill_policy=fill-latest.

An additional configuration param, coord_global_policy, accepts a Boolean value, by default `false`. When it is set to true, it will mask out the x/y portion of the node report, leaving only the lat/lon portion. The manner in which the lat/lon is set is determined by the crossfill_policy configuration parameter.

2.7 Processing Alternate Navigation Solutions

Under normal circumstances, node reports are generated reflecting the current navigation solution as defined by the incoming NAV_* variables. The pNodeReporter application can handle the case where the vehicle also publishes an alternate navigation solution, as defined by a sister set of incoming MOOS variables separate from the NAV_* variables. In this case pNodeReporter will monitor both sets of variables and may generate *two* node reports on each iteration. The following two configuration parameters are needed to activate this capability:

```
alt_nav_prefix = <prefix>      // example: NAV_GT_
alt_nav_name   = <node-name>   // example: _GT
alt_nav_group  = <group-name>  // example: ground_truth (Post R.19.8.x)
```

The configuration parameter, alt_nav_prefix, names a prefix for the alternate incoming navigation variables. For example, alt_nav_prefix=NAV_GT_ would result in pNodeReporter subscribing for NAV_GT_X, NAV_GT_Y and so on. A separate vehicle state would be maintained internally based on this alternate set of navigation information and a second node report would be generated. The alt_nav_group allows the alt nav node report to have a different group name than the baseline node report. If not specified, the group names will match. This feature was introduced after Release 19.8.x.

A second node report would be published under the same MOOS variable, NODE_REPORT_LOCAL, but the NAME component of the report would be distinct based on the value provided in the alt_nav_name parameter. If a name is provided that does not begin with an underscore character, that name is used. If the name does begin with an underscore, the name used in the report is the otherwise configured name of the vehicle plus the suffix.

2.8 Publishing Node Reports in JSON Format

Post Release 24.8, node reports may optionally be published in JSON format. Either (a) the NODE_REPORT_LOCAL message can be made in JSON format, or (b) a second MOOS variable can be specified to be additionally published in JSON format. The deserialization function, converting node

report strings into a C++ record, will auto-detect the format. All apps in MOOS-IvP ingesting node reports use this function.

To publish NODE_REPORT_LOCAL in JSON format, simply use:

```
  json_report = true
```

If the value of the argument is any string other than `true`, or `false`, case insensitive, the value is interpreted to mean a MOOS variable to which a second node report will be published in JSON format. For example, consider the below configuration:

```
  json_report = NODE_REPORT_JSON
```

Both the comma-separated-pair (CSP) format, and JSON format will be published:

```
NODE_REPORT_JSON = {"NAME":"abe","SPD":1.9,"HDG":190.45,"DEP":0,"LAT":43.82472523,
    "LON":-70.33059595,"TYPE":"kayak","COLOR":"yellow",
    "MODE":"MODE@ACTIVE:LOITERING","ALLSTOP":"clear",
    "INDEX":113,"YAW":-1.753119,"TIME":17398343683.8,"LENGTH":4}

NODE_REPORT_LOCAL = NAME=abe,SPD=1.9,HDG=190.45,DEP=0,LAT=43.82472523,
    LON=-70.33059595,TYPE=kayak,COLOR=yellow,
    MODE=MODE@ACTIVE:LOITERING,ALLSTOP=clear,
    INDEX=113,YAW=-1.753119,TIME=17398343683.8,LENGTH=4
```

3 The Optional Blackout Interval Option

Under normal circumstances, the pNodeReporter application will post a node report once per iteration, the gap between postings being determined solely by the app_tick parameter (Figure 2). However, there are times when it is desirable to add an artificial delay between postings. Node reports are typically only useful as information sent to another node, or to a shoreside computer rendering fielded vehicles, and there are often dropped node report messages due to the uncertain nature of communications in the field, whether it be acoustic communications, WiFi, or satellite link.

Applications receiving node reports usually implement provisions that take dropped messages into account. A collision-avoidance or formation-following behavior, or a contact manager, may extrapolate a contact position from its last received position and trajectory. A shoreside command-and-control GUI such as pMarineViewer may render an interpolation of vehicle positions between node reports. To test the robustness of applications needing to deal with dropped messages, a way of simulating the dropped messages is desired. One way is to add this to the simulation version of whatever communications medium is being used. For example, there is an acoustic communications simulator where the dropping of messages may be simulated, where the probability of a drop may even be tied to the range between vehicles. Another way is to simply simulate the dropped message at the source, by adding delay to the posting of reports by pNodeReporter.

By setting the blackout_interval parameter, pNodeReporter may be configured to ensure that a node report is not posted until *at least* the duration specified by this parameter has elapsed, as shown in Figure 3.

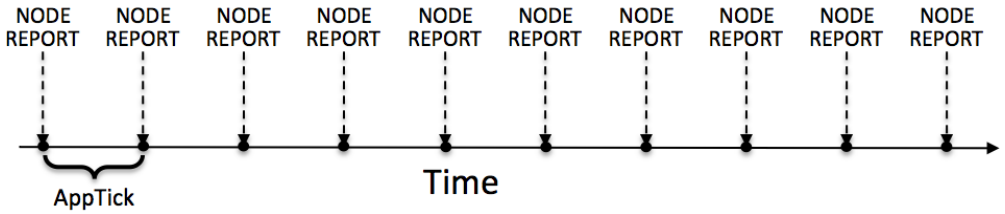

Figure 2: **Normal schedule of node report postings**: The `pNodeReporter` application will post node reports once per application iteration. The duration of time between postings is directly tied to the frequency at which `pNodeReporter` is configured to run, as set by the standard MOOS `AppTick` parameter.

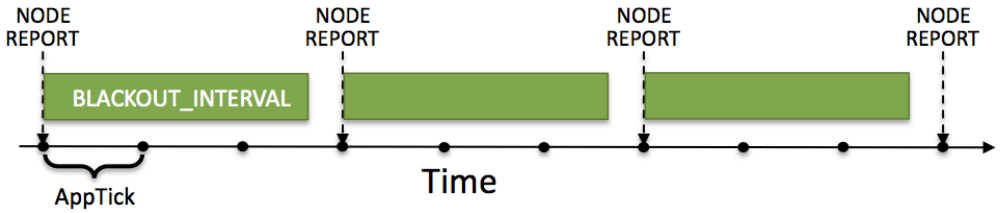

Figure 3: **The optional blackout interval parameter:** The schedule of node report postings may be altered by the setting the `BLACKOUT_INTERVAL` parameter. Reports will not be posted until at least the time specified by the blackout interval has elapsed since the previous posting.

An element of unpredictability may be added by specifying a value for the blackout_variance parameter. This parameter is given in seconds and defines an interval $[-t, t]$ from which a value is chosen with uniform probability, to be added to the duration of the blackout interval. This variation is re-calculated after each interval determination. The idea is depicted in Figure 4.

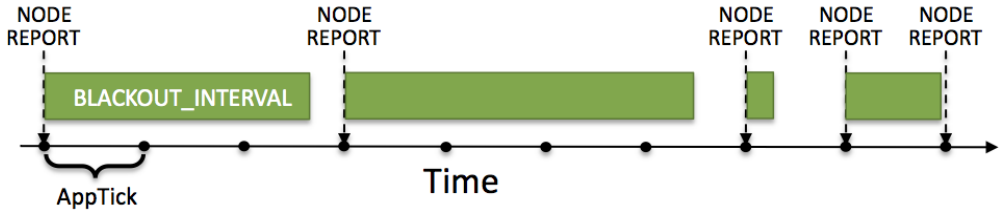

Figure 4: **Blackout intervals with varying duration:** The duration of a blackout interval may be configured to vary randomly within a user-specified range, specified in the blackout_variance parameter.

Message dropping is typically tied semi-predictably to characteristics of the environment, such as range between nodes, water temperature or platform depth, an so on. This method of simulating

dropped messages captures none of that. It is however simple and allows for easily proceeding with the testing of applications that need to deal with the dropped messages.

4 Configuration Parameters for pNodeReporter

The following parameters are defined for pNodeReporter. A more detailed description is provided in other parts of this section. Parameters having default values are indicated so in parentheses.

Listing 4.1: Configuration Parameters for pNodeReporter.

allow_color_change:	If true, then incoming requests via the MOOS variable NODE_COLOR_CHANGE will be respected. The default is `false`.
alt_nav_group:	Group associated with node reports generated for alternate nav node reports. Introduced after Release 19.8.x. Section 2.7.
alt_nav_name:	Node name in posting alternate nav reports. Section 2.7.
alt_nav_prefix:	Source for processing alternate nav reports. Section 2.7.
blackout_interval:	Minimum duration, in seconds, between reports. The default is zero. Section 3.
blackout_variance:	Variance in uniformly random blackout duration. Legal values: any non-negative value. The default is zero. Section 3.
coord_policy_global:	If true (default is false), then only lat/lon coordinates are published in the node report; no local x/y coordinates. Section 2.6.
crossfill_policy:	Policy for handling local versus global nav reports (`"literal"`). Section 2.6.
extrap_enabled:	If `true` (the default is `false`), then a node report is not generated on every iteration, but can instead be held back for some time, assuming the consumers of the node reports are capable of extrapolating the vehicle position. This can reduce network traffic and log file size, but must be used with care and when sure that consumers are capable of extrapolation. Section 2.4.
extrap_hdg_thresh:	If extrapolation is enabled, this parameter, in degrees, determines a threshold for posting a new node report. If the vehicle heading has changed by degrees set here, a new node report must be posted. The default is 1 degree. Section 2.4.
extrap_max_gap:	If extrapolation is enabled, this parameter, in seconds, determines a threshold for posting a new node report. If node report has not been published in the time set here, a new node report must be posted. The default is 5 seconds. Section 2.4.
extrap_pos_thresh:	If extrapolation is enabled, this parameter, in meters, determines a threshold for posting a new node report. If the vehicle position has changed by this distance, a new node report must be posted. The default is `0.25` meters. Section 2.4.

hdg_error: A delta heading applied to the heading field for each node report, relative to the otherwise reported heading from NAV_HEADING. This feature is used solely to test how downstream consumers/apps that ingest node reports handle a situation where the reported heading and course of ground vary substantially. The default value is 0.

json_report: If `true`, the node report will be published in JSON format. If set to something other than true, e.g., FOOBAR, a second node report will be published, in JSON format, to FOOBAR, and the normal node report in CSP format will be published as normal. Section 2.8.

nav_grace_period: A duration, in seconds, before a run warning is posted due to mission navigation information, e.g, NAV_LAT or NAV_LONG.

node_report_output: MOOS variable used for the node report (NODE_REPORT_LOCAL). Section 2.1.

nohelm_threshold: Seconds after which a quiet helm is reported as AWOL. Legal values: any non-negative value. The default is 5 seconds. Section 2.2.

paused: If true, posting of reports is suspended. The default is `"false"`. With release 15.3.

plat_report_output: The Platform report MOOS variable. Legal values: conventions for MOOS variable names. The default is PLATFORM_REPORT_LOCAL. Section 6.

plat_report_input: A component of the optional platform report. Section 6.

platform_aspect: A custom param=value pair to be include in all node reports. Section 2.3.1.

platform_beam: The reported beam length (from side-to-side at the widest point) of the platform in meters. Legal values: any non-negative value. The default is zero. Section 2.5.

platform_color: A hint on how to render the vehicle in a GUI application like pMarineViewer or alogview. Introduced in Release 16.5. Section 2.5.

platform_group: Specify a group name to be include in all node reports. By default empty and nod include in report messages. Can be useful if simulating competitions with teams.

platform_length: The reported length of the platform in meters. Legal values: any non-negative value. The default is zero. Section 2.5.

platform_type: The reported type of the platform. Legal values: any string. The default is `"unknown"`. Section 2.5.

platform_transparency: Node reports may include a transparency hint betwen [0,1], with zero being nearly transparent. Some viewers, e.g., pMarineViewer may respect this hint. The default value is 1. When transparency is ommitted from a node report, it is assumed be 1.

report_cog: If true, node reports will include course over ground (COG) information calculated based on the angle between the current position and previous position. The default is `false`.

rider: A custom augmentation of the node report can be configured, by registering for one or more MOOS variables and reformatting the output and tagging onto (riding) the end of a node report. Post 24.8. See Section 2.3.2.

terse_reports: If true, do not include `YAW` in node reports.

An Example MOOS Configuration Block

An example MOOS configuration block is provided in Listing 2 below. To see an example MOOS configuration block from the console, enter the following:

```
$ pNodeReporter --example or -e
```

This will show the output shown in Listing 2 below.

Listing 4.2: Example configuration of pNodeReporter.

```
1  ============================================================
2  pNodeReporter Example MOOS Configuration
3  ============================================================
4  Blue lines: Default configuration
5
6  ProcessConfig = pNodeReporter
7  {
8    AppTick   = 4
9    CommsTick = 4
10
11   // Configure key aspects of the node
12   platform_type      = glider   // or {uuv,auv,ship,kayak}
13   platform_length    = 8        // meters.  Range [0,inf)
14
15   // Configure optional blackout functionality
16   blackout_interval  = 0        // seconds. Range [0,inf)
17
18    // Configure the optional platform report summary
19   plat_report_input  = COMPASS_HEADING, gap=1
20   plat_report_input  = GPS_SAT, gap=5
21   plat_report_input  = WIFI_QUALITY, gap=1
22   plat_report_output = PLATFORM_REPORT_LOCAL
23
24   // Configure the MOOS variable containg the node report
25   node_report_output = NODE_REPORT_LOCAL
26
27   // Threshold for conveying an absense of the helm
28   nohelm_threshold   = 5        // seconds
39
30   // Policy for filling in missing lat/lon from x/y or v.versa
31   // Valid policies: [literal], fill-empty, use-latest, global
32   crossfill_policy   = literal
33
34   // Configure monitor/reporting of dual nav solution
35   alt_nav_prefix     = NAV_GT
36   alt_nav_name       = _GT
37 }
```

5 Publications and Subscriptions for pNodeReporter

The interface for pNodeReporter, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ pNodeReporter --interface or -i
```

5.1 Variables Published by pNodeReporter

The primary output of pNodeReporter to the MOOSDB is the node report and the optional platform report:

- APPCAST: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.
- NODE_REPORT_LOCAL: Primary summary of the node's navigation and helm status. Section 2.1.
- PLATFORM_REPORT_LOCAL: Optional summary of certain platform characteristics. Section 2.5.

5.2 Variables Subscribed for by pNodeReporter

Variables subscribed for by pNodeReporter are summarized below. A more detailed description of each variable follows. In addition to these variables, any MOOS variable that the user requests to be included in the optional PLATFORM_REPORT will also be automatically subscribed for.

- APPCAST_REQ: A request to generate and post a new apppcast report, with reporting criteria, and expiration.
- IVPHELM_STATE: A indicator of the helm state produced by pHelmIvP, e.g., "PARK", "DRIVE" "DISABLED", or "STANDBY".
- IVPHELM_ALLSTOP: A indicator of the helm allstop produced by pHelmIvP, e.g., "clear", or a reason why the vehicle is at zero speed.
- IVPHELM_SUMMARY: A summary report produced by the IvP Helm (pHelmIvP).
- NAV_X: The ownship vehicle position on the x axis of local coordinates.
- NAV_Y: The ownship vehicle position on the y axis of local coordinates.
- NAV_LAT: The ownship vehicle position on the y axis of global coordinates.
- NAV_LONG: The ownship vehicle position on the x axis of global coordinates.
- NAV_HEADING: The ownship vehicle heading in degrees.
- NAV_YAW: The ownship vehicle yaw in radians.
- NAV_SPEED: The ownship vehicle speed in meters per second.
- NAV_DEPTH: The ownship vehicle depth in meters.
- PNR_PAUSE: Allows the paused state to be set or toggled. Acceptable values are "true" "false" "toggle". With release 15.3.

If pNodeReporter is configured to handle a second navigation solution as described in Section 2.7, the corresponding additional variables described in that section will also be automatically subscribed for.

5.3 Command Line Usage of pNodeReporter

The pNodeReporter application is typically launched as a part of a batch of processes by pAntler, but may also be launched from the command line by the user. The basic command line usage for the pNodeReporter application is the following:

Listing 5.3: Command line usage for the pNodeReporter application.

```
Usage: pNodeReporter file.moos [OPTIONS]

Options:
  --alias=<ProcessName>
      Launch pNodeReporter with the given process name
      rather than pNodeReporter.
  --example, -e
      Display example MOOS configuration block.
  --help, -h
      Display this help message.
  --version,-v
      Display the release version of pNodeReporter.
```

6 The Optional Platform Report Feature

The pNodeReporter application allows for the optional reporting of another user-specified list of information. This report is made by posting the PLATFORM_REPORT_LOCAL variable. An alternative variable name may be used by setting the PLAT_REPORT_SUMMARY configuration parameter. This report may be configured by specifying one or more components in the pNodeReporter configuration block, of the following form:

```
plat_report_input = <variable>, gap=<duration>, alias=<variable>
```

If no component is specified, then no platform report will be posted. The `<variable>` element specifies the name of a MOOS variable. This variable will be automatically subscribed for by pNodeReporter and included in (not necessarily all) postings of the platform report. If the variable BODY_TEMP is specified, a component of the report may contain "BODY_TEMP=98.6". An alias for a MOOS variable may be specified. For example, `alias=T`, for the BODY_TEMP component would result in "T=98.6" in the platform report instead.

How often is the platform report posted? Certainly it will not be posted any more often than the apptick parameter allows, but it may be posted far more infrequently depending on the user configuration and how often the values of its components are changing. The platform report is posted only when one or more of its components requires a re-posting. A component requires a re-posting only if (a) its value has changed, *and* (b) the time specified by its gap setting has elapsed since the last platform report that included that component. When a PLATFORM_REPORT_LOCAL posting is made, only components that required a posting will be included in the report.

The wide variation in configurations of the platform report allow for reporting information about the node that may be very specific to the platform, not suitable for a general-purpose node report. As an example, consider a situation where a shoreside application is running to monitor the platform's battery level and whether or not the payload compartment has suffered a breach, i.e., the presence of water is detected inside. A platform report could be configured as follows:

```
plat_report_input = ACME_BATT_LEVEL, gap=300, alias=BATTERY_LEVEL
plat_report_input = PAYLOAD_BREACH
```

This would result in an initial posting of:

```
PLATFORM_REPORT_LOCAL = "platform=alpha,utc_time=1273510720.99,BATTERY_LEVEL=97.3,
                        PAYLOAD_BREACH=false"
```

In this case, the platform uses batteries made by the ACME Battery Company and the interface to the battery monitor happens to publish its value in the variable ACME_BATT_LEVEL, and the software on the shoreside that monitors all vehicles in the field accepts the generic variable BATTERY_LEVEL, so the alias is used. It is also known that the ACME battery monitor output tends to fluctuate a percentage point or two on each posting, so the platform report is configured to include a battery level component no more than once every five minutes, (gap=300). The MOOS process monitoring the indication of a payload breach is known to have few false alarms and to publish its findings in the variable PAYLOAD_BREACH. Unlike the battery level which has frequent minor fluctuations and degrades slowly, the detection of a payload breach amounts to the flipping of a Boolean value and needs to be conveyed to the shoreside as quickly as possible. Setting gap=0, the default, ensures that a platform report is posted on the very next iteration of pNodeReporter, presumably to be read by a MOOS process controlling the platform's outgoing communication mechanism.

7 An Example Platform Report Configuration Block for pNodeReporter

Listing 4 below shows an example configuration block for pNodeReporter where an extensive platform report is configured to report information about the autonomous kayak platform to support a "kayak dashboard" display running on a shoreside computer. Most of the components in the platform report are specific to the autonomous kayak platform, which is precisely why this information is included in the platform report, and not the node report.

Listing 7.4: An example pNodeReporter configuration block.

```
1  //--------------------------------------------------------------
2  // pNodeReporter config block
3
4  ProcessConfig = pNodeReporter
5  {
6    AppTick  = 2
7    CommsTick = 2
8
9    platform_type       = KAYAK
10   platform_length     = 3.5     // Units in meters
11   nohelm_thresh       = 5       // The default
12   blackout_interval   = 0       // The default
13   blackout_variance   = 0       // The default
14
15   node_report_output  = NODE_REPORT_LOCAL        // The default
16   plat_report_output  = PLATFORM_REPORT_LOCAL    // The default
17
18   plat_report_input = COMPASS_PITCH, gap=1
19   plat_report_input = COMPASS_HEADING, gap=1
20   plat_report_input = COMPASS_ROLL, gap=1
```

```
21    plat_report_input = DB_UPTIME, gap=1
22    plat_report_input = COMPASS_TEMPERATURE, gap=1, alias=COMPASS_TEMP
23    plat_report_input = GPS_MAGNETIC_DECLINATION, gap=10, alias=MAG_DECL
24    plat_report_input = GPS_SAT, gap=5
25    plat_report_input = DESIRED_RUDDER, gap=0.5
26    plat_report_input = DESIRED_HEADING, gap=0.5
27    plat_report_input = DESIRED_THRUST, gap=0.5
28    plat_report_input = GPS_SPEED, gap=0.5
29    plat_report_input = DESIRED_SPEED, gap=0.5
30    plat_report_input = WIFI_QUALITY, gap=0.5
31    plat_report_input = WIFI_QUALITY, gap=1.0
32    plat_report_input = MOOS_MANUAL_OVERRIDE, gap=1.0
33  }
```

8 Measuring the Odometry Extent Per Mission Hash

A new feature, *after* Release 22.8, is introduced to enable pNodeReporter to post useful information to be logged with the vehicle alog file. This coincides with the introduction of a *mission hash*. The mission hash is published by the shoreside and shared to all vehicles and logged. It has the form:

```
MISSION_HASH = mhash=221119-1255K-NEAR-MILK,utc=1668880538.69
```

The posting contains both the hash itself, 221119-1255K-NEAR-MILK, and the UTC timestamp when the hash was created on the shoreside. The first part of the hash reflects the time and date, 241119-1255, Nov 19th, 2024, 12:55. The second part of the hash is comprised of randomly configured words that are easier to remember. The final two-word part of the hash can be used as an alias in some applications.

The shoreside generates the hash, and it cannot be ruled out that the shoreside generates multiple hashes for a single vehicle alog file. This can happen for example if the shoreside app, e.g., pMarineViewer, generating the mission hash, is re-started during the vehicle deployment. In this case the vehicle alog file may have multiple mission hashes, leading to confusion. This is where pNodeReporter can help.

pNodeReporter will register for MISSION_HASH and use an odometer to track the maximum vehicle extent associated with this hash. The *extent* is the maximum distance from the odometer starting point, i.e., the farthest linear distance from the starting point at any time in the mission. The node reporter will publish to PNR_MHASH whenever this maximum extent has reached a new max value. If the MISSION_HASH value should change mid-mission, the PNR_MHASH value will also change.

If the mission hash changes mid-mission, we may see a set of alog entries from pMarineViewer along the lines of:

```
14.853   PNR_MHASH  mhash=221119-1412M-GOLD-EPIC,ext=2.6
15.874   PNR_MHASH  mhash=221119-1412M-GOLD-EPIC,ext=3.9
16.897   PNR_MHASH  mhash=221119-1412M-GOLD-EPIC,ext=5.4
22.538   PNR_MHASH  mhash=221119-1412M-GOLD-EPIC,ext=6.9
23.560   PNR_MHASH  mhash=221119-1412M-GOLD-EPIC,ext=8.3
24.587   PNR_MHASH  mhash=221119-1412M-GOLD-EPIC,ext=9.5
26.119   PNR_MHASH  mhash=221119-1412M-GOLD-EPIC,ext=10.6
177.068  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=12.1
178.093  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=13.5
179.117  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=14.9
180.129  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=16.3
181.159  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=17.8
182.172  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=19.2
183.207  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=20.4
184.228  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=21.8
218.643  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=23.2
219.682  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=24.3
220.693  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=25.4
222.219  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=26.9
223.762  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=28.3
225.317  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=29.8
226.876  PNR_MHASH  mhash=221119-1413E-MAIN-KHAN,ext=31.1
```

In this case the mission hash was changed sometime between the offset times of 26.119 and 177.068. When pNodeReporter noticed that the max extent distance accumulated while the mission hash was "MAIN-KHAN" had exceeded 10.6, a new PNR_MHASH posting was made.

In a nutshell, if there was unfortunately more than one mission hash received by a vehicle and logged in a single alog file, this information can help determine which mission hash was the "real" one. It should always be the hash associated with the final posting to PNR_MHASH. And this information can be obtained using a few aloggrep switches:

```
$ aloggrep file.alog --final --no_report PNR_MHASH
226.876  PNR_MHASH    pNodeReporter   mhash=221119-1413E-MAIN-KHAN,ext=31.1
```

Or, to isolate the hash itself:

```
$ aloggrep file.alog --final --format=val --subpat=mhash PNR_MHASH
221119-1413E-MAIN-KHAN
```

The latter example is essentially the whole point of this feature: to enable aloggrep to determine the most likely single mission hash for a give alog file, even if multiple hashes exist. This tool can then be used as a component of other scripts that are performing post-mission log file archiving tasks.

uSimMarineV22: Vehicle Simulation

July 2022

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/appdocs/app_usimmarine_v22`

1 Overview

The `uSimMarineV22` application is a 3D vehicle simulator that updates vehicle state, position and trajectory, based on the present actuator values and prior vehicle state. This app is the successor to the `uSimMarine` app. Released in 2022, `uSimMarineV22` is a backward compatible superset of `uSimMarine`, with the following notable changes or additions:

- Embeded PID controller: The user has the option to embed a PID controller within the simulator for near-zero latency between PID output and simulator position updates. This allows for substantial increases in maximum time warp. Where previous missions had a maximum time warp of say 50x realtime, 200x to 300x realtime has been observed.
- Worm holes: A worm hole a region in the operation area that will consume an vehicle, and instantly shift its position to another region in the operating area. This facilitates missions where traffic patterns of several contacts are used for testing collision avoidance.
- Sailing: The simulator will apply basic sailing effects on the vehicle given (a) a polar plot describing vehicle speed relative to a given wind direction and magnitude, (b) incoming updates to the current wind direction and magnitude.

The typical usage scenario has a single instance of uSimMarineV22 associated with each simulated vehicle, as shown in Figure 1.

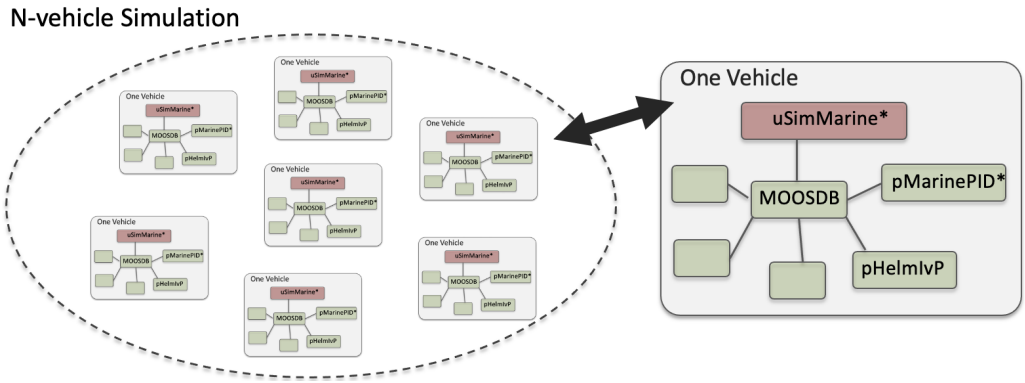

Figure 1: **Typical** uSimMarineV22 **Usage:** In an N-vehicle simulation, an instance if uSimMarineV22 is used for each vehicle. Each simulated vehicle typically has its own dedicated MOOS community. The IvP Helm (pHelmIvP) publishes high-level control decisions. The PID controller (**pMarinePID**) converts the high-level control decisions to low-level actuator decisions. Finally the simulator (uSimMarineV22) reads the low-level actuator postings to produce a new vehicle position.

This style of simulation can be contrasted with simulators that simulate a comprehensive set of aspects of the simulation, including multiple vehicles, and aspects of the environment and communications. The uSimMarineV22 simulator simply focuses on a single vehicle. It subscribes for the vehicle navigation state variables:

- NAV_X, NAV_Y, NAV_SPEED, NAV_HEADING, and NAV_DEPTH

as well as the actuator values

- DESIRED_RUDDER, DESIRED_THRUST, and DESIRED_ELEVATOR.

The simulator accommodates a notion of external drifts applied to the vehicle to crudely simulate current or wind. These drifts may be set statically or may be changing dynamically by other

MOOS processes. The simulator also may be configured with a simple geo-referenced data structure representing a field of water currents.

Under typical UUV payload autonomy operation, the simulator and pMarinePID MOOS modules would not be present. The vehicle's native controller would handle the role of pMarinePID, and the vehicle's native navigation system (and the vehicle itself) would handle the role of the simulator.

2 Configuration Parameters for uSimMarineV22

The following parameters are defined for uSimMarineV22. A more detailed description is provided in other parts of this section. Parameters having default values are indicated so in parentheses below.

Listing 2.1: Configuration Parameters for uSimMarineV22.

buoyancy_rate:	Rate, in meters per second, at which vehicle floats to surface at zero speed. The default is zero. Section 5.4.
current_field:	A file containing the specification of a current field.
current_field_active:	If true, simulator uses the current field if specified.
default_water_depth:	Default value for local water depth for calculating altitude (0).
drift_vector:	A pair of external drift values, direction and magnitude.
rotate_speed:	An external rotational speed in degrees per second (0).
drift_x:	An external drift value applied in the x direction (0).
drift_y:	An external drift value applied in the y direction (0).
max_acceleration:	Maximum rate of vehicle acceleration in m/s^2 (0.5).
max_deceleration:	Maximum rate of vehicle deceleration in m/s^2 (0.5).
max_depth_rate:	Maximum rate of vehicle depth change, meters per second. The defauls is 0.5. Section 5.4.
max_depth_rate_speed:	Vehicle speed at which max depth rate is achievable (2.5). Section 5.4.
prefix:	Prefix of MOOS variables published. The default is USM_.
sim_pause:	If true, the simulation is paused. The default is false.
start_depth:	Initial vehicle depth in meters. The default is zero. Section 4.
start_heading:	Initial vehicle heading in degrees. The default is zero. Section 4.
start_pos:	A full starting position and trajectory specification. Section 4.
start_speed:	Initial vehicle speed in meters per second. The default is zero. Section 4.
start_x:	Initial vehicle x position in local coordinates. The default is zero. Section 4.
start_y:	Initial vehicle y position in local coordinates. The default is zero. Section 4.
thrust_factor:	A scalar correlation between thrust and speed. The default is 20.
thrust_map:	A mapping between thrust and speed values. Section 8.
thrust_reflect:	If true, negative thrust is simply opposite positive thrust. The default is false. Section 8.

turn_loss: A range $[0, 1]$ affecting speed lost during a turn. The default is 0.85.

turn_rate: A range $[0, 100]$ affecting vehicle turn radius, e.g., 0 is an infinite turn radius. The default is 70.

An Example MOOS Configuration Block

An example MOOS configuration block is provided in Listing 2 below. This can also be obtained from a terminal window with:

```
$ uSimMarineV22 --example or -e
```

Listing 2.2: Example configuration of the uSimMarineV22 application.

```
1  ============================================================
2  uSimMarineV22 Example MOOS Configuration
3  ============================================================
4
5  ProcessConfig = uSimMarineV22
6  {
7    AppTick   = 4
8    CommsTick = 4
9
10   start_x       = 0
11   start_y       = 0
12   start_heading = 0
13   start_speed   = 0
14   start_depth   = 0
15   start_pos     = x=0, y=0, speed=0, heading=0, depth=0
16
17   drift_x       = 0
18   drift_y       = 0
19   rotate_speed  = 0
20   drift_vector  = 0,0      // heading, magnitude
21
22   buoyancy_rate        = 0.025 // meters/sec
23   max_acceleration     = 0     // meters/sec^2
24   max_deceleration     = 0.5   // meters/sec^2
25   max_depth_rate       = 0.5   // meters/sec
26   max_depth_rate_speed = 2.0   // meters/sec
27
28   sim_pause            = false // or {true}
29   dual_state           = false // or {true}
30   thrust_reflect       = false // or {true}
31   thrust_factor        = 20    // range [0,inf)
32   turn_rate            = 70    // range [0,100]
33   thrust_map           = 0:0, 20:1, 40:2, 60:3, 80:5, 100:5
34 }
```

3 Publications and Subscriptions for uSimMarineV22

The interface for uSimMarineV22, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ usimMarine --interface or -i
```

3.1 Variables Published by uSimMarineV22

The primary output of uSimMarineV22 to the MOOSDB is the full specification of the updated vehicle position and trajectory, along with a few other pieces of information:

- APPCAST: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.
- BUOYANCY_REPORT:
- TRIM_REPORT:
- USM_ALTITUDE: The updated vehicle altitude in meters if water depth known.
- USM_DEPTH: The updated vehicle depth in meters. Section 5.4.
- USM_DRIFT_SUMMARY: A summary of the current total external drift.
- USM_HEADING: The updated vehicle heading in degrees.
- USM_HEADING_OVER_GROUND: The updated vehicle heading over ground.
- USM_LAT: The updated vehicle latitude position.
- USM_LONG: The updated vehicle longitude position.
- USM_RESET_COUNT: The number of time the simulator has been reset.
- USM_SPEED: The updated vehicle speed in meters per second.
- USM_SPEED_OVER_GROUND: The updated speed over ground.
- USM_X: The updated vehicle x position in local coordinates.
- USM_Y: The updated vehicle y position in local coordinates.
- USM_YAW: The updated vehicle yaw in radians.

An example USM_DRIFT_SUMMARY string: `"ang=90, mag=1.5, xmag=90, ymag=0"`.

3.2 Variables Subscribed for by uSimMarineV22

The uSimMarineV22 application will subscribe for the following MOOS variables:

- APPCAST_REQ: A request to generate and post a new apppcast report, with reporting criteria, and expiration. See the documentation on Appcasting.
- DESIRED_THRUST: The thruster actuator setting, $[-100, 100]$.
- DESIRED_RUDDER: The rudder actuator setting, $[-100, 100]$.
- DESIRED_ELEVATOR: The depth elevator setting, $[-100, 100]$.
- USM_SIM_PAUSED: Simulation pause request, either `true` or `false`.
- USM_CURRENT_FIELD: If `true`, a configured current field is active.
- USM_BUOYANCY_RATE: Dynamically set the zero-speed float rate.

- ROTATE_SPEED: Dynamically set the external rotational speed.
- DRIFT_X: Dynamically set the external drift in the x direction.
- DRIFT_Y: Dynamically set the external drift in the y direction.
- DRIFT_VECTOR: Dynamically set the external drift direction and magnitude.
- DRIFT_VECTOR_ADD: Dynamically modify the external drift vector.
- DRIFT_VECTOR_MULT: Dynamically modify the external drift vector magnitude.
- USM_RESET: Reset the simulator with a new position, heading, speed and depth.
- WATER_DEPTH: Water depth at the present vehicle position.

Each iteration, after noting the changes in the navigation and actuator values, it posts a new set of navigation state variables in the form of USM_X, USM_Y, USM_SPEED, USM_HEADING, USM_DEPTH.

3.3 Command Line Usage of uSimMarineV22

The uSimMarineV22 application is typically launched as a part of a batch of processes by pAntler, but may also be launched from the command line by the user. The basic command line usage for the uSimMarineV22 application is the following:

Listing 3.3: Command line usage for the uSimMarineV22 application.

```
Usage: uSimMarineV22 file.moos [OPTIONS]

Options:
  --alias=<ProcessName>
      Launch uSimMarineV22 with the given process name
      rather than uSimMarineV22.
  --example, -e
      Display example MOOS configuration block.
  --help, -h
      Display this help message.
  --version,-v
      Display the release version of uSimMarineV22.
```

4 Setting the Initial Vehicle Position, Pose and Trajectory

The simulator is typically configured with a vehicle starting position, pose and trajectory given by the following five configuration parameters:

- start_x
- start_y
- start_heading
- start_speed
- start_depth

The position is specified in local coordinates in relation to a local datum, or $(0, 0)$ position. This datum is specified in the .moos file at the global level. The heading is specified in degrees and corresponds to the direction the vehicle is pointing. The initial speed and depth by default are zero, and are often left unspecified in configuration. Alternatively, the same five parameters may be set with the start_pos parameter as follows:

```
start_pos = x=100, y=150, speed=0, heading=45, depth=0
```

The simulator can also be reset at any point during its operation, by posting to the MOOS variable USM_RESET. A posting of the following form will reset the same five parameters as above:

```
USM_RESET = x=200, y=250, speed=0.4, heading=135, depth=10
```

This has been useful in cases where the objective is to observe the behavior of a vehicle from several different starting positions, and an external MOOS script, e.g., uTimerScript, is used to reset the simulator from each of the desired starting states.

5 Propagating the Vehicle Speed, Heading, Position and Depth

The vehicle position is updated on each iteration of the uSimMarineV22 application, based on (a) the previous vehicle state, (b) the elapsed time since the last update, ΔT, (c) the current actuator values, DESIRED_RUDDER, DESIRED_THRUST, and DESIRED_ELEVATOR, and (d) several parameter settings describing the vehicle model.

For simplicity, this simulator updates the vehicle speed, heading, position and depth in sequence, in this order. For example, the position is updated after the heading is updated, and the new position update is made as if the new heading were the vehicle heading for the entire ΔT. The error introduced by this simplification is mitigated by running uSimMarineV22 with a fairly high MOOS AppTick value keeping the value of ΔT sufficiently small.

5.1 Propagating the Vehicle Speed

The vehicle speed is propagated primarily based on the current value of thrust, which is presumably refreshed upon each iteration by reading the incoming mail on the MOOS variable DESIRED_THRUST. To simulate a small speed penalty when the vehicle is conducting a turn through the water, the new thrust value may also be affected by the current rudder value, referenced by the incoming MOOS variable DESIRED_RUDDER. The newly calculated speed is also dependent on the previously noted speed noted by the incoming MOOS variable NAV_SPEED, and the settings to the two configuration parameters MAX_ACCELERATION and MAX_DECELERATION.

The algorithm for updating the vehicle speed proceeds as:

1. Calculate $v_{i(\texttt{RAW})}$, the new raw speed based on the thrust.
2. Calculate $v_{i(\texttt{TURN})}$, an adjusted and potentially lower speed, based on the raw speed, $v_{i(\texttt{RAW})}$, and the current rudder angle, DESIRED_RUDDER.
3. Calculate $v_{i(\texttt{FINAL})}$, an adjusted and potentially lower speed based on $v_{i(\texttt{TURN})}$, compared to the prior speed. If the magnitude of change violates either the max acceleration or max deceleration settings, then the new speed is clipped appropriately.
4. Set the new speed to be $v_{i(\texttt{FINAL})}$, and use this new speed in the later updates on heading, position and depth.

Step 1: In the first step, the new speed is calculated by the current value of thrust. In this case the *thrust map* is consulted, which is a mapping from possible thrust values to speed values. The thrust

map is configured with the THRUST_MAP configuration parameter, and is described in detail in Section 8.

$$v_{i_{(\texttt{RAW})}} = \texttt{THRUST\_MAP}(\texttt{DESIRED\_THRUST})$$

Step 2: In the second step, the calculated speed is potentially reduced depending on the degree to which the vehicle is turning, as indicated by the current value of the MOOS variable DESIRED_RUDDER. If it is not turning, it is not diminished at all. The adjusted speed value is set according to:

$$v_{i_{(\texttt{TURN})}} = v_{i_{(\texttt{RAW})}} * (1 - (\frac{|\texttt{RUDDER}|}{100} * \texttt{TURN\_LOSS}))$$

The configuration parameter turn_loss is a value in the range of $[0, 1]$. When set to zero, there is no speed lost in any turn. When set to 1, there is a 100% speed loss when there is a maximum rudder. The default value is 0.85.

Step 3: In the last step, the candidate new speed, $v_{i_{(\texttt{TURN})}}$, is compared with the incoming vehicle speed, v_{i-1}. The elapsed time since the previous simulator iteration, ΔT, is used to calculate the acceleration or deceleration implied by the new speed. If the change in speed violates either the min_acceleration, or max_acceleration parameters, the speed is adjusted as follows:

$$v_{i_{(\texttt{FINAL})}} = \begin{cases} v_{i-1} + (\texttt{MAX\_ACCELERATION} * \Delta T) & \frac{(v_{i_{(\texttt{TURN})}} - v_{i-1})}{\Delta T} > \texttt{MAX\_ACCELERATION}, \\ v_{i-1} - (\texttt{MAX\_DECELERATION} * \Delta T) & \frac{(v_{i-1} - v_{i_{(\texttt{TURN})}})}{\Delta T} > \texttt{MAX\_DECELERATION}, \\ v_{i_{(\texttt{TURN})}} & \text{otherwise.} \end{cases}$$

Step 4: The final speed from the previous step is posted by the simulator as USM_SPEED, and is used the calculations of position and depth, described next.

5.2 Propagating the Vehicle Heading

The vehicle heading is propagated primarily based on the current RUDDER value which is refreshed upon each iteration by reading the incoming mail on the MOOS variable DESIRED_RUDDER, and the elapsed time since the simulator previously updated the vehicle state, ΔT. The change in heading my also be influenced by the THRUST value from the MOOS variable DESIRED_THRUST, and may also factor an external rotational speed.

The algorithm for updating the new vehicle heading proceeds as:

1. Calculate $\Delta\theta_{i(\texttt{RAW})}$, the new raw change in heading influenced only by the current rudder value.
2. Calculate $\Delta\theta_{i(\texttt{THRUST})}$, an adjusted change in heading, based on the raw change in heading, $\Delta\theta_{i(\texttt{RAW})}$, and the current THRUST value.
3. Calculate $\Delta\theta_{i(\texttt{EXTERNAL})}$, an adjusted change in heading considering external rotational speed.
4. Calculate θ_i, the final new heading based on the calculated change in heading and the previous heading, and converted to the range of $[0, 359]$.

Step 1: In the first step, the new heading is calculated by the current RUDDER value:

$$\Delta\theta_{i(\texttt{RAW})} = \texttt{RUDDER} * \frac{\texttt{TURN\_RATE}}{100} * \Delta T$$

The TURN_RATE is an uSimMarineV22 configuration parameter with the allowable range of $[0, 100]$. The default value of this parameter is 70, chosen in part to be consistent with the performance of the simulator prior to this parameter being exposed to configuration. A value of 0 would result in the vehicle never turning, regardless of the rudder value.

Step 2: In the second step the influence of the current vehicle thrust (from the MOOS variable DESIRED_THRUST) may be applied to the change in heading. The magnitude of the change of heading is adjusted to be greater when the thrust is greater than 50% and less when the thrust is less than 50%.

$$\Delta\theta_{i(\texttt{THRUST})} = \theta_{i(\texttt{RAW})} * (1 + \frac{|\texttt{THRUST}| - 50}{50})$$

The direction in heading change is then potentially altered based on the sign of the THRUST:

$$\Delta\theta_{i(\texttt{THRUST})} = \begin{cases} -\Delta\theta_{i(\texttt{THRUST})} & \texttt{THRUST} < 0, \\ \Delta\theta_{i(\texttt{THRUST})} & \text{otherwise.} \end{cases}$$

Step 3: In the third step, the change in heading may be further influenced by an external rotational speed. This speed, if present, would be read at the outset of the simulator iteration from either the configuration parameter rotate_speed, or dynamically from the MOOS variable ROTATE_SPEED. The updated value is calculated as follows:

$$\Delta\theta_{i(\texttt{EXTERNAL})} = \theta_{i(\texttt{THRUST})} + (\texttt{ROTATE\_SPEED} * \Delta T)$$

Step 4: In final step, the final new heading is set based on the previous heading and the change in heading calculated in the previous three steps. If needed, the value of the new heading is converted to its equivalent heading in the range $[0, 359]$.

$$\theta_i = \texttt{heading360}(\theta_{i-1} + \Delta\theta_{i(\texttt{EXTERNAL})})$$

The simulator then posts this value to the MOOSDB as USM_HEADING.

5.3 Propagating the Vehicle Position

The vehicle position is propagated primarily based on the newly calculated vehicle heading and speed, the previous vehicle position, and the elapsed time since updating the previous vehicle position, ΔT.

The algorithm for updating the new vehicle position proceeds as:

1. Calculate the vehicle heading and speed used for updating the new vehicle position, with the heading converted into radians.
2. Calculate the new positions, x_i and y_i, based on the heading, speed and elapsed time.
3. Calculate a possibly revised new position, factoring in any external drift.

Step 1: In the first step, the heading value, $\bar{\theta}$, and speed value, $\bar{v}$ used for calculating the new vehicle position is set averaging the newly calculated values with their prior values:

$$\bar{v} = \frac{(v_i + v_{i-1})}{2} \tag{1}$$

$$\bar{\theta} = \text{atan2}(s, c)$$

where s and c are given by:

$$s = sin(\theta_{i-1}\pi/180) + sin(\theta_i\pi/180)$$
$$c = cos(\theta_{i-1}\pi/180) + cos(\theta_i\pi/180)$$

The above calculation of the heading average handles the issue of angle wrap, i.e., the average of 359 and 1 is zero, not 180.

Step 2: The vehicle x and y position is updated by the following two equations:

$$x_i = x_{i-1} + sin(\bar{\theta}) * \bar{v} * \Delta\mathtt{T}$$
$$y_i = y_{i-1} + cos(\bar{\theta}) * \bar{v} * \Delta\mathtt{T}$$

The above is calculated keeping in mind the difference in convention used in marine navigation where zero degrees is due North and 90 degrees is due East. That is, the mapping is as follows from marine to traditional trigonometric convention: $0° \rightarrow 90°$, $90° \rightarrow 0°$, $180° \rightarrow 270°$, $270° \rightarrow 180°$.

Step 3: The final step adjusts the x, and y position from above, taking into consideration any external drift that may be present. This drift includes both the drift that may be directed from the incoming MOOS variables as described in Section 7. The drift components below are also a misnomer since they are provided in units of meters per second.

$$x_i = x_i + \texttt{EXTERNAL\_DRIFT\_X} * \Delta\mathtt{T} \tag{2}$$
$$y_i = y_i + \texttt{EXTERNAL\_DRIFT\_Y} * \Delta\mathtt{T} \tag{3}$$

5.4 Propagating the Vehicle Depth

Depth change in uSimMarineV22 is simulated based on a few input parameters. The primary parameter that changes from one iteration to the next is the ELEVATOR actuator value, from the MOOS variable DESIRED_ELEVATOR. On any given iteration the new vehicle depth, z_i, is determined by:

$$z_i = z_{i-1} + (\dot{z}_i * \Delta t)$$

The new vehicle depth is altered by the *depth change rate*, $\dot{z}_i$, applied to the elapsed time, Δt, which is roughly equivalent to the apptick interval set in the uSimMarineV22 configuration block. The depth change rate on the current iteration is determined by the vehicle speed and the ELEVATOR actuator value, and by the following three vehicle-specific simulator configuration parameters that allow for some variation in simulating the physical properties of the vehicle. The buoyancy_rate, for simplicity, is given in meters per second where positive values represent a positively buoyant vehicle. The max_depth_rate, and max_depth_rate_speed parameters determine the function(s) shown in Figure 2. The vehicle will have a higher depth change rate at higher speeds, up to some maximum speed where the speed no longer affects the depth change rate. The actual depth change rate then depends on the elevator and vehicle speed.

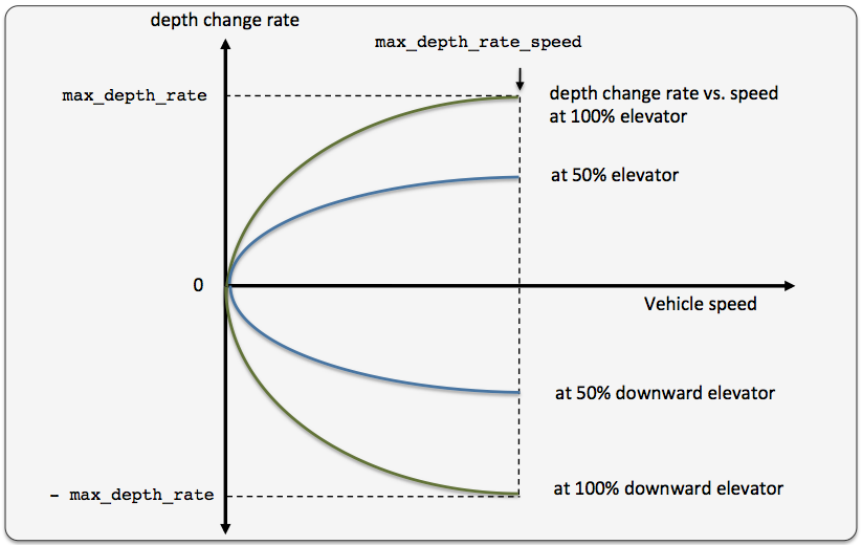

Figure 2: The relationship between the rate of depth change rate, given a current vehicle speed. Different elevator settings determine unique curves as shown.

The value of the depth change rate, $\dot{v}_i$, is determined as follows:

$$\dot{z}_i = \left(\frac{\bar{v}}{\texttt{MAX\_DEPTH\_RATE\_SPEED}}\right)^2 * \frac{\texttt{ELEVATOR}}{100} * \texttt{MAX\_DEPTH\_RATE} + \texttt{BUOYANCY\_RATE} \tag{4}$$

Both fraction components in the above equation are clipped to $[-1, 1]$. When the vehicle is in reverse thrust and has a negative speed, this equation still holds. However, a vehicle would likely not have a depth change rate curve symmetric between positive and negative vehicle speeds. By default the value of buoyancy_rate is set to 0.025, slightly positively buoyant, max_depth_rate is set to 0.5, and max_depth_rate_speed is set to 2.0. The prevailing buoyancy rate may be dynamically adjusted by a separate MOOS application publishing to the variable BUOYANCY_RATE.

6 Propagating the Vehicle Altitude

The vehicle altitude is base solely on the current vehicle depth and the depth of the water at the current vehicle position. If nothing is known about the water depth, then USM_ALTITUDE is not published. The simulator may be configured with a default water depth:

```
default_water_depth  = 100
```

This will allow the simulator to produce some altitude information if needed for testing consumers of USM_ALTITUDE information. Furthermore, the simulator subscribes for water depth information in the variable USM_WATER_DEPTH which could conceivably be produced by another MOOS application with access to bathymetry data and the vehicle's navigation position.

7 Simulation of External Drift

When the simulator updates the vehicle position as in equations provided in Section 5.3, it factors a possible external drift in the x and y directions, in the term EXTERNAL_DRIFT_X, and EXTERNAL_DRIFT_X respectively. The external drift may have two distinct components; a drift applied generally, and a drift applied due to a current field configured with an external file correlating drift vectors to local x and y positions. These drifts may be set in one of three ways discussed next.

7.1 External X-Y Drift from Initial Simulator Configuration

An external drift may be configured upon startup by either specifying explicitly the drift in the x and y direction, or by specifying a drift magnitude and direction. Figure 3 shows two external drifts each with the appropriate configuration using either the drift_x and drift_y parameters or the single drift_vector parameter:

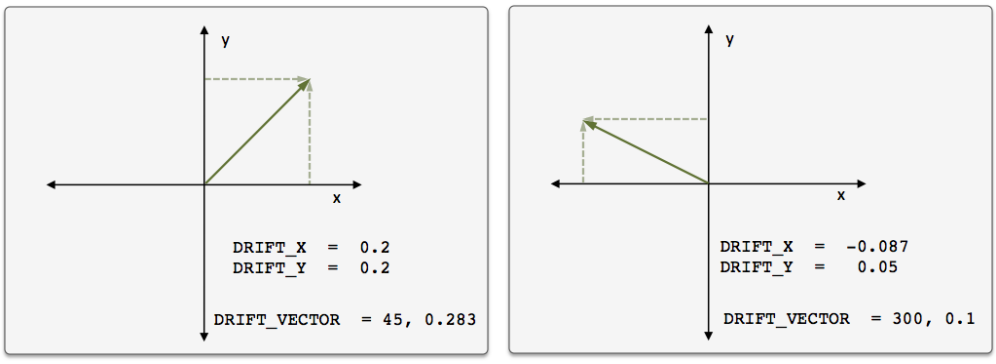

Figure 3: **External Drift Vectors**: Two drift vectors each configured with either the drift_x and drift_y configuration parameters or their equivalent single drift_vector parameter.

If, for some reason, the user mistakenly configures the simulator with both configuration styles, the configuration appearing last in the configuration block will be the prevailing configuration. If uSimMarineV22 is configured with these parameters, these external drifts will be applied on the very first iteration and all later iterations unless changed dynamically, as discussed next.

7.2 External X-Y Drift Received from Other MOOS Applications

External drifts may be adjusted dynamically by other MOOS applications based on any criteria wished by the user and developer. The uSimMarineV22 application registers for the following MOOS variables in this regard: DRIFT_X, DRIFT_Y, DRIFT_VECTOR, DRIFT_VECTOR_ADD, DRIFT_VECTOR_MULT. The first three variables simply override the previously prevailing drift, set by either the initial configuration or the last received mail concerning the drift.

By posting to the USM_DRIFT_VECTOR_MULT variable the *magnitude* of the prevailing vector may be modified with a single multiplier such as:

```
DRIFT_VECTOR_MULT = 2
DRIFT_VECTOR_MULT = -1
```

The first MOOS posting above would double the size of the prevailing drift vector, and the second example would reverse the direction of the vector. The DRIFT_VECTOR_ADD variable describes a drift vector to be *added* to the prevailing drift vector. For example, consider the prevailing drift vector shown on the left in Figure 3, with the following MOOS mail received by the simulator:

```
DRIFT_VECTOR_ADD = "262.47, 15.796"
```

The resulting drift vector would be the vector shown on the right in Figure 3. This interface opens the door for the scripting changes to the drift vector like the one below, that crudely simulate a gust of wind in a given direction that builds up to a certain magnitude and dies back down to a net zero drift.

```
DRIFT_VECTOR_ADD = 137, 0.25
DRIFT_VECTOR_ADD = 137, 0.25
DRIFT_VECTOR_ADD = 137, 0.25
DRIFT_VECTOR_ADD = 137, 0.25
DRIFT_VECTOR_ADD = 137, 0.25
DRIFT_VECTOR_ADD = 137, -0.25
DRIFT_VECTOR_ADD = 137, -0.25
DRIFT_VECTOR_ADD = 137, -0.25
DRIFT_VECTOR_ADD = 137, -0.25
DRIFT_VECTOR_ADD = 137, -0.25
```

The above style script was described in the documentation for uTimerScript, where the uTimerScript utility was used to simulate wind gusts in random directions with random magnitude. The DRIFT_* interface may also be used by any third party MOOS application simulating things such as ocean or wind currents.

8 The ThrustMap Data Structure

A *thrust map* is a data structure that may be used to simulate a non-linear relationship between thrust and speed. This is configured in the `uSimMarineV22` configuration block with the `thrust_map` parameter containing a comma-separated list of colon-separated pairs. Each element in the comma-separated list is a single mapping component. In each component, the value to the left of the colon is a thrust value, and the other value is a corresponding speed. The following is an example mapping given in string form, and rendered in Figure 4.

```
thrust_map = "-100:-3.5, -75:-3.2, -10:-2, 20:2.4, 50:4.2, 80:4.8, 100:5"
```

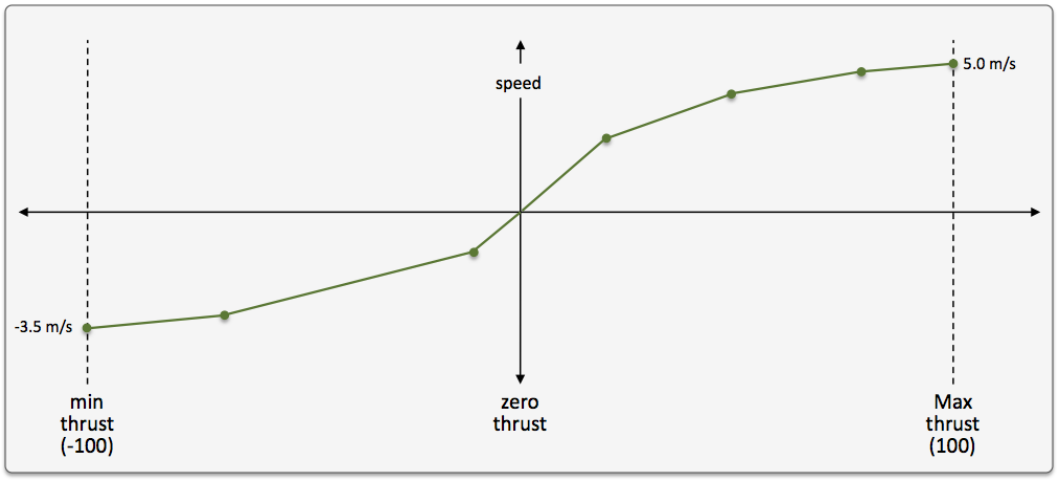

Figure 4: **A Thrust Map**: The example thrust map was defined by seven mapping points in the string "-100:-3.5, -75:-3.2, -10:-2, 20:2.4, 50:4.2, 80:4.8, 100:5".

8.1 Automatic Pruning of Invalid Configuration Pairs

The thrust map has an immutable domain of $[-100, 100]$, indicating 100% forward and reverse thrust. Mapping pairs given outside this domain will simply be ignored. The thrust mapping must also be monotonically increasing. This follows the intuition that more positive thrust will not result in the vehicle going slower, and likewise for negative thrust. Since the map is configured with a sequence of pairs as above, a pair that would result in a non-monotonic map is discarded. All maps are created as if they had the pair `0:0` given explicitly. Any pair provided in configuration with zero as the thrust value will ignored; zero thrust always means zero speed. Therefore, the following map configurations would all be equivalent to the map configuration above and shown in Figure 4:

```
thrust_map = -120:-5, -100:-3.5, -75:-3.2, -10:-2, 20:2.4, 50:4.2, 80:4.8, 100:5.0, 120:6
thrust_map = -100:-3.5, -75:-3.2, -10:-2, 20:2.4, 50:4.2, 80:4.8, 90:4, 100:5.0
thrust_map = -100:-3.5, -75:-3.2, -10:-2, 0:0, 20:2.4, 50:4.2, 80:4.8, 100:5.0
thrust_map = -100:-3.5, -75:-3.2, -10:-2, 0:1, 20:2.4, 50:4.2, 80:4.8, 100:5.0
```

In the first case, the pairs `"-120:-5"` and `"120:6"` would be ignored since they are outside the $[-100, 100]$ domain. In the second case, the pair `"90:4"` would be ignored since its inclusion would entail a non-monotonic mapping given the previous pair of `"80:4.8"`. In the third case, the pair `"0:0"` would be effectively ignored since it is implied in all map configurations anyway. In the fourth case, the pair `"0:1"` would be ignored since a mapping from a non-zero speed to zero thrust is not permitted.

8.2 Automatic Inclusion of Implied Configuration Pairs

Since the domain $[-100, 100]$ is immutable, the thrust map is altered a bit automatically when or if the user provides a configuration without explicit mappings for the thrust values of -100 or 100. In this case, the missing mapping becomes an implied mapping. The mapping `100:v` is added where `v` is the speed value of the closest point. For example, the following two configurations are equivalent:

```
thrust_map = -75:-3.2, -10:-2, 20:2.4, 50:4.2, 80:4.8
thrust_map = -100:-3.2, -75:-3.2, -10:-2, 20:2.4, 50:4.2, 80:4.8, 100:4.8
```

8.3 A Shortcut for Specifying the Negative Thrust Mapping

For convenience, the mapping of positive thrust values to speed values can be used in reverse for negative thrust values. This is done by configuring `uSimMarineV22` with `thrust_reflect=true`, which is false by default. If `thrust_reflect` is false, then a speed of zero is mapped to all negative thrust values. If `thrust_reflect` is true, but the user nevertheless provides a mapping for a negative thrust in a thrust map, then the `thrust_reflect` directive is simply ignored and the thrust map is used instead. For example, the following two configurations are equivalent:

```
thrust_map = -100:-5, -80:-4.8, -50:-4.2, -20:-2.4, 20:2.4, 50:4.2, 80:4.8, 100:5
```

and

```
thrust_map = 20:2.4, 50:4.2, 80:4.8, 100:5
thrust_reflect = true
```

8.4 The Inverse Mapping - From Speed To Thrust

Since a thrust map only permits configurations resulting in a non-monotonic function, the inverse also holds (almost) as a valid mapping from speed to thrust. We say ”almost” because there is ambiguity in cases where there is one or more plateau in the thrust map as in:

```
thrust_map = -75:-3.2, -10:-2, 20:2.4, 50:4.2, 80:4.8
```

In this case a speed of 4.8 maps to any thrust in the range $[80, 100]$. To remove such ambiguity, the thrust map, as implemented in a C++ class with methods, returns the lowest magnitude thrust in such cases. A speed of 4.8 (or 5 for that matter), would return a thrust value of 80. A speed of

−3.2 would return a thrust value of −75. The motivation for this way of disambiguation is that if a thrust value of 80 and 100, both result in the same speed, one would always choose the setting that conserves less energy. Reverse mappings are not used by the uSimMarineV22 application, but may be of use in applications responsible for posting a desired thrust given a desired speed, as with the pMarinePID application.

8.5 Default Behavior of an Empty or Unspecified ThrustMap

If uSimMarineV22 is configured without an explicit thrust_map or thrust_reflect configuration, the default behavior is governed as if the following two lines were actually included in the uSimMarineV22 configuration block:

```
thrust_map     = 100:5
thrust_reflect = false
```

The default thrust map is rendered in Figure 5.

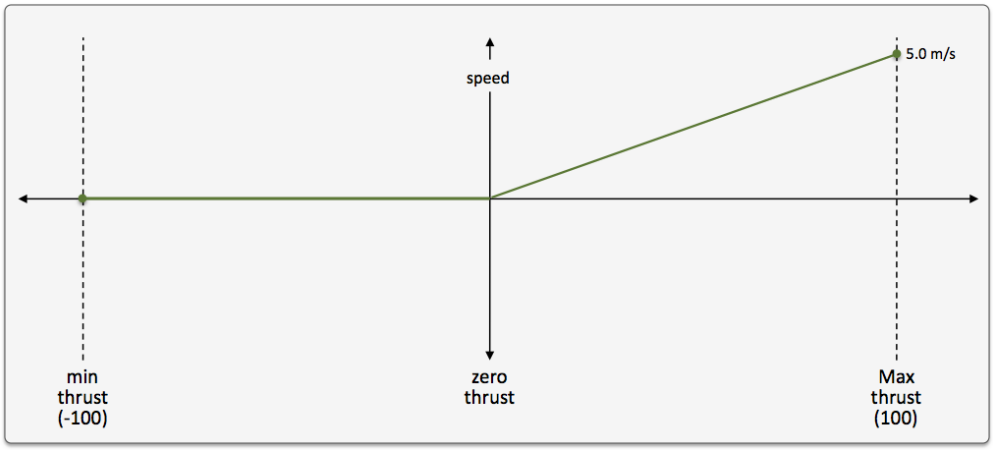

Figure 5: **The Default Thrust Map**: This thrust map is used if no explicit configuration is provided.

This default configuration was chosen for its reasonableness, and to be consistent with the behavior of prior versions of uSimMarineV22 where the user did not have the ability to configure a thrust map.

9 Wormholes

A *wormhole* connects two points in space whereby a vehicle entering from one direction immediately emerges at the opposite end. While the existence in nature is just a theory, their existence in `uSimMarineV22` is a feature now available as of July 2022.

9.1 Wormhole Motivation

A goal of certain simulutions is to test vehicle performance, given some initial starting conditions. If the goal is to test performace over say thousands of variations of the mission, the user has roughly two options. (a) Repeatedly launch the mission with the right conditions, shut down the mission, note the results, and repeat. (b) Within a single mission, repeatedly re-create the starting conditions, let the event of interest unfold again, note the results and repeat.

The advantage of (a) is that the mission is short and "clean". There are no phases of the simulation where the vehicles are re-setting or driving back to their starting positions. The drawback comes with the overhead of repeatedly starting, killing, post-processing a simulation for each test of the event of interest. The advantage of (b) is that many events are observed for a single mission start, kill, post-process cycle. The drawbacks of (b) are that, between events, vehicles need to drive to re-set the conditions for the event of interest. Consider the mission in Figure 6 below. A single vehicle traverse a travel lane, where several contacts are crossing the lane. After the initial traversals, each vehicle drives back to re-set the starting conditions.

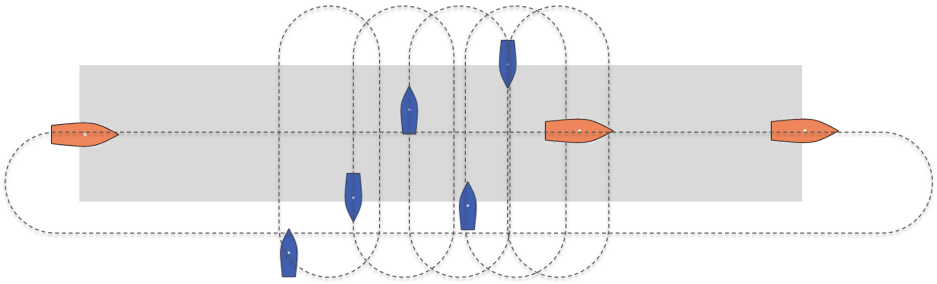

Figure 6: **Simulation with No Wormhole**: A single vehicle, ownship, traverses a travel lane, avoiding collisions with several vehicles which may be crossing the lane. When ownship reaches the end of the lane it traverses back to the beginning of the lane, perhaps encountering contacts outside the intended context for testing. Likewise, the contacts also repeatedly turn around to re-enter the travel lane as foils for testing ownship collision avoidance.

Wormholes can be used in simulations like the one shown above in Figure 6, to create a variation like the one shown in Figure 7. The orange test vehicle, when it reaches the end of the travel lane, will enter the wormhole to instantly re-appear at the same end of the travel lane it came from. The contacts in this example also do not need to turn around (and potentially take evasive action between each other during the turns).

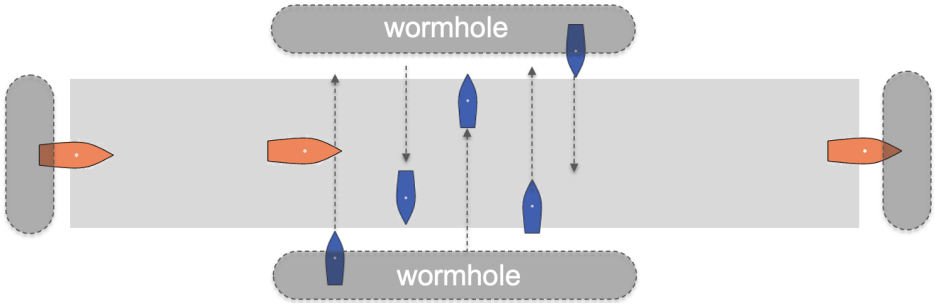

Figure 7: **Simulation with Wormhole**: The single orange vehicle, ownship, traverses a travel lane, entering a wormhole at the end of the lane, and re-emerging at the start of the lane. Contacts crossing the travel lane each enter a wormhole on the destination side, to then immediately re-emerge at the originating side. All vehicles continuously progress in one direction with no need for turning around.

9.2 Wormhole Configuration

A wormhole is configured with a pair of polygons, one polygon is called the `madrid_poly`, and the other is the `weber_poly`. The cities of Madrid (Spain) and Weber (New Zealand), are Antipode cities, lying exactly or nearly exactly at opposite coorindates on the globe.

The below three configuration lines declare the entry and exit polygons and the direction in which the wormhole is configure. The `tag` component allows multiple wormholes to be confgured, each with unique tag.

```
wormhole = tag=one, madrid_poly={format=ellipse,x=80,y=-180,degs=0,major=160,minor=20,pts=20}
wormhole = tag=one, weber_poly={format=ellipse,x=80,y=0,degs=0,major=160,minor=20,pts=20}
wormhole = tag=one, connection=from_madrid
wormhole = tag=one, id_change=true
wormhole = tag=one, delay=0
```

The last two components, `id_change=true` and `delay=3`, are related to features not yet implement. Three features are yet to be implemented:

- `connection`: Currently the wormhole only works in one declared direction. Bi-directional wormholes will be implemented to support, for example, the bi-directional traffic shown in Figure 7.
- `id_change`: Currently the vehicle name does not change as it passes through the wormhole. At times it may be useful if the name does change, as a manner of testing the operation of the contact manager which needs to handle memory management over many ephemeral contacts over a long mission.
- `delay`: Currently a vehicle entering a wormhole will instantly appear on the other